Primer módulo Python que consume un servicio externo (Bloque 5) / De script suelto a módulo: funciones, JSON y orden

✓ **Hecho:** Ver

UNIDAD U1.3 · Python aplicado para integración con IA

De script suelto a módulo: funciones, JSON y orden

Con tu entorno de desarrollo completo y reproducible, empieza el trabajo que le da sentido a todo ese setup: escribir código real.

> Esta **unidad** tiene un objetivo concreto y acotado, que vas a construir a lo largo de tres bloques: un módulo **Python**, dentro de tu repositorio, que consume un servicio externo, procesa la respuesta en formato **JSON**, y gestiona los errores que puedan aparecer en el camino.

Esta primera lección sienta las bases de cómo se organiza ese código, antes de escribir la primera línea.

🧩 Qué es exactamente un script suelto, y qué es un módulo

> Un **script suelto** es código escrito de arriba hacia abajo, pensado para ejecutarse una sola vez, de principio a fin, sin ninguna estructura que permita reutilizar partes de él en otro contexto.

Es perfectamente válido para probar una idea rápida, y probablemente ya escribiste alguno así en algún

momento.

El problema aparece cuando ese código necesita usarse de más de una forma: si mañana quieres consumir el mismo servicio pero con otro parámetro, o integrar esa misma lógica dentro de una aplicación más grande, un script suelto obliga a copiar y pegar código, y cada copia es una oportunidad más para que algo se desincronice del original.

> **Un módulo es la respuesta a ese problema**: un archivo **Python** organizado como una colección de funciones con responsabilidades claras, pensado para ser importado y reutilizado desde otras partes de tu **proyecto**, no solo ejecutado de una vez.

La diferencia no es de cantidad de código, es de intención:



Script

Un **script** resuelve algo puntual y se descarta.



Módulo

Un **módulo** se diseña para durar y para que otras partes de tu **aplicación** dependan de él con confianza.

El **módulo** que vas a construir en esta unidad va a vivir en la carpeta **"src"** de tu repositorio, y es, literalmente, la primera pieza permanente del proyecto que vas a entregar en el **Gate 3**.

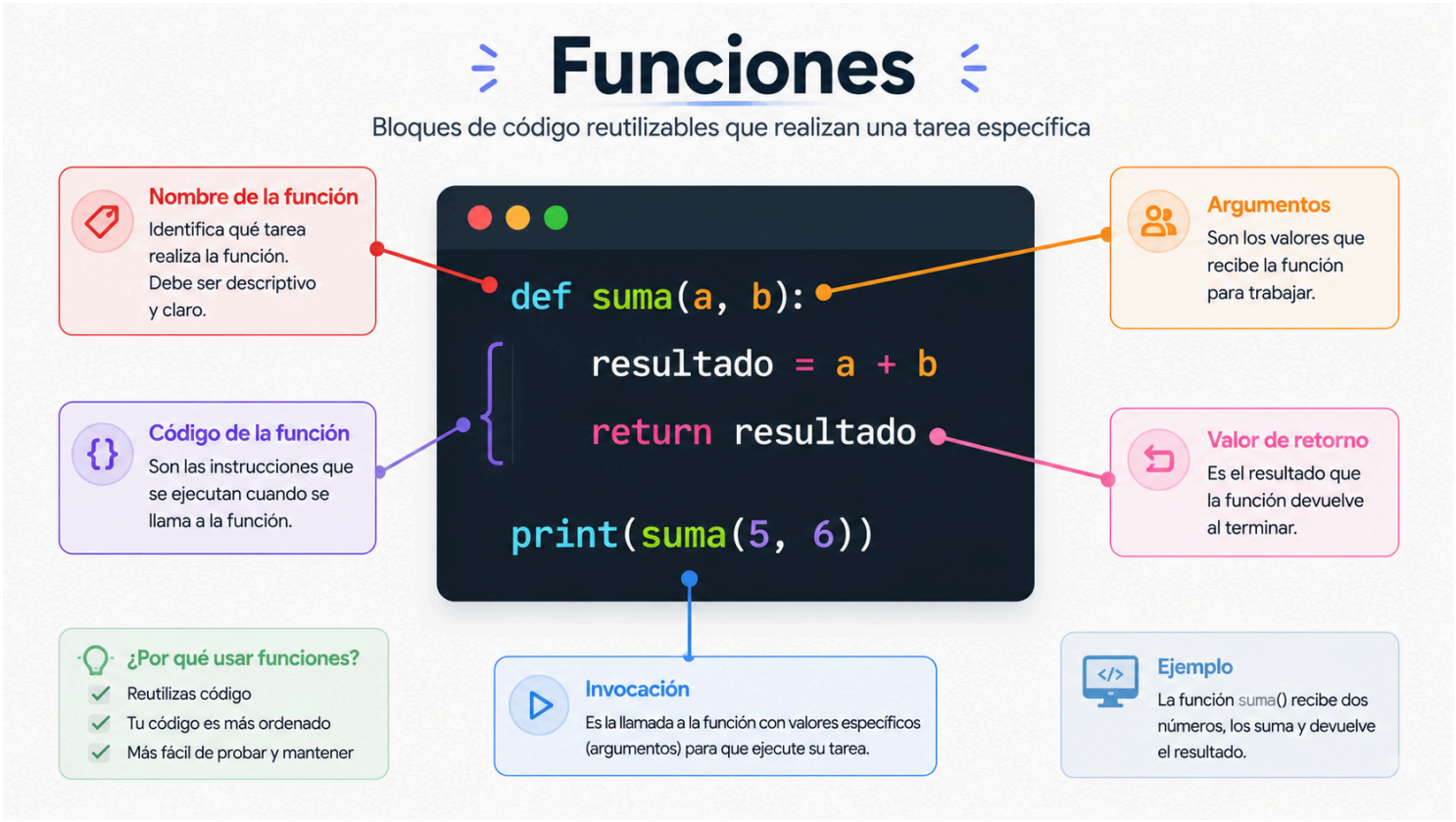
Concretamente, dentro de la estructura que ya trae tu plantilla base:

```
tu-repositorio/
├── src/        # aquí va a vivir tu módulo
├── docs/
├── tests/
├── .env
├── .env.example
└── requirements.txt
```

⚙️ Funciones: la unidad básica de organización

La forma en que un módulo logra ser reutilizable es dividiendo su trabajo en funciones: bloques de código con un nombre, una entrada definida (los parámetros que recibe) y una salida definida (lo que devuelve), pensados para hacer una sola cosa y hacerla bien.

Una función que hace demasiadas cosas a la vez (armar la petición, enviarla, procesar la respuesta, y decidir qué hacer si algo falla, todo junto) es difícil de leer, difícil de probar, y difícil de reutilizar en un contexto distinto al que se pensó originalmente.



El criterio que vas a aplicar durante esta unidad es simple de enunciar aunque no siempre simple de lograr: si al describir qué hace una función necesitas usar la palabra "y" varias veces, probablemente esa función debería dividirse en dos o más.

> 🎯 Este mismo criterio de responsabilidad única es, además, uno de los cuatro criterios de la rúbrica de **calidad de código** que se usa para revisar tu **módulo** al final de esta **unidad**, así que conviene interiorizarlo desde esta primera lección, no dejarlo para el final.

📦 JSON: el formato que vas a ver en cada respuesta

Ya viste **JSON** de pasada en el **Bloque 4**, al revisar cómo se lee la respuesta de una petición HTTP.

Ahora que vas a procesar esa respuesta con código, conviene mirarlo con más atención. **JSON** organiza la información en pares de nombre y valor (algo como el nombre de un dato y su contenido), y esos pares pueden anidarse unos dentro de otros: un valor puede ser, a su vez, otro conjunto de pares, o una lista de varios elementos.

Cuando tu código en **Python** recibe una respuesta en este formato, la convierte automáticamente en las estructuras de datos nativas del lenguaje: los pares de nombre y valor se convierten en un diccionario, y las listas se convierten en listas de **Python**.

Todo el trabajo de procesar una respuesta, en el fondo, consiste en navegar ese diccionario (y esas listas dentro de él) hasta llegar al dato específico que te interesa.

💡 **Un ejemplo pequeño ayuda a verlo concretamente**. Una respuesta JSON como esta:

```
{
   "texto": "Hola, soy una respuesta generada",
   "detalles": {
     "modelo": "nombre-del-modelo",
     "tokens": 12
   }
 }
```

Llega a tu código **Python** convertida en un diccionario, y para llegar al texto que te interesa navegas ese diccionario por sus nombres, igual que harías con cualquier otro diccionario:

```
respuesta["texto"]
 # "Hola, soy una respuesta generada"

 respuesta["detalles"]["modelo"]
 # "nombre-del-modelo"
```

> ☝ Los nombres exactos de los campos ("**texto**", "**detalles**", "**modelo**") **cambian según el proveedor**: esto es solo un ejemplo genérico para mostrar el mecanismo, no la estructura real de ninguna **API** en particular.

Puesto en un script mínimo y ejecutable, con la conversión explícita de JSON a diccionario, se ve así:

```
import json

respuesta_json = '{"texto": "Hola, soy una respuesta generada", "detalles": {"modelo": "nom
respuesta = json.loads(respuesta_json)

print(respuesta["texto"])
print(respuesta["detalles"]["modelo"])
```

Su salida exacta en consola:

```
Hola, soy una respuesta generada
nombre-del-modelo
```



Dos errores con los que probablemente te vas a topar más adelante, ya con tu propio código, vale la pena reconocerlos desde ahora:



"KeyError"

Aparece si le pides al diccionario un nombre de campo que no existe (por ejemplo, **respuesta["texti"]** en el ejemplo de arriba, con una letra de más). Revisa que el nombre que estás usando coincida exactamente con el que trae la respuesta real de tu proveedor, no con el de este ejemplo genérico.



"ModuleNotFoundError"

En esta unidad casi nunca es un problema de cómo organizaste tus carpetas: normalmente significa que te falta instalar una librería externa (con **"pip install"**, con tu entorno virtual activo) o que la instalaste con el entorno virtual equivocado.

Vas a ver ambos casos en código real, no solo en teoría, cuando construyas tu propio módulo más adelante en este mismo bloque.

 El orden que vas a seguir en este bloque y en los siguientes

Esta unidad avanza en un orden deliberado, y conviene que lo tengas presente para no sentir que cada lección empieza de cero:



En la siguiente lección de este mismo bloque vas a ver, la construcción de un **módulo** de consumo y un error de red provocado a propósito, para mostrar cómo se maneja de forma controlada.



Más adelante en este mismo bloque vas a construir tu propia primera versión de este módulo, todavía simple, consumiendo un servicio de prueba.



En el Bloque 6 vas a tomar ese resultado y refactorizarlo: convertirlo de un bloque de código más bien plano a funciones reutilizables, organizadas dentro de tu módulo.

En el Bloque 7 vas a incorporarle manejo de excepciones y validación de entradas, y ese es el punto en el que tu módulo cumple con el entregable completo de esta unidad.

Nada de este orden es azaroso: primero se construye algo que funciona, después se organiza mejor, y al final se blinda contra lo que puede salir mal. Intentar hacer las tres cosas a la vez, desde la primera línea, suele producir código más confuso que si se hacen en ese orden, un paso a la vez.

Siguiente lección

Bloque 5 · Manejo de errores

Con esta base conceptual clara (qué es un **módulo**, por qué las **funciones** importan, y cómo se estructura una respuesta **JSON**), la siguiente lección de este mismo bloque muestra todo esto funcionando junto, en una demostración en vivo.

Última modificación: jueves, 24 de septiembre de 2026, 12:27